

Software Test Plan (STP) – HuffZip: File Compression Tool

Project: HuffZip – File Compression Tool (Basic ZIP Implementation using Huffman Coding), Problem Statement #14

Version: 1.0

Authors: Team 14 – Pranavi Sistla (PES1UG24CS338), Pavan Kumar H K (PES1UG24CS319), Rachit Upadhyay (PES1UG24CS355), Inchara K Kuppall (PES1UG25CS818)

Date: 04-10-2026

Status: Draft

1. Introduction

Purpose: This document defines the test plan for HuffZip v1.0, a command-line file compression tool written in C/C++ that uses Huffman coding. It describes the test objectives, scope, strategy, environment, schedule, responsibilities, test cases and traceability to the SRS.

Scope: Testing covers compression, decompression, archive header handling, CRC32 integrity checking, statistics reporting, error handling, edge cases (empty and single-symbol files), security of archive parsing, performance, memory usage, compression ratio and portability. Compatibility with standard ZIP tools, GUI and encryption are excluded (see Section 4).

References: HuffZip SRS v1.0; HuffZip Software Architecture & Design Specification v1.0; IEEE 829 / IEEE 29119-3 (test documentation); Canterbury Corpus (corpus.canterbury.ac.nz); OWASP guidance on file-upload and archive handling.

Definitions: SRS (Software Requirements Specification), SAD (Software Architecture & Design Specification), RTM (Requirements Traceability Matrix), CRC32 (32-bit Cyclic Redundancy Check), Huffman coding (variable-length prefix coding based on symbol frequency), Round-trip (compress then decompress and compare), ASan (AddressSanitizer), RSS (Resident Set Size), .huf (HuffZip archive file).

2. Test Items

- CLI / Argument Parser module (huffzip main program)
- File I/O module (chunked binary read/write)
- Frequency Analyzer module
- Huffman Tree Builder and Code Table Generator modules
- Encoder / Bit Writer module
- Decoder / Bit Reader module
- Archive Header Writer/Reader module
- Integrity Checker (CRC32) module
- Error Handler module

3. Features to be Tested

Features mapped to SRS requirement IDs:

Requirement ID	Feature / Requirement
HZ-F-001	Compress a single input file into a .huf archive using Huffman coding
HZ-F-002	Decompress a .huf archive and restore the original file
HZ-F-003	Build the byte-frequency table and Huffman tree / code table
HZ-F-004	Write archive header: magic "HUFZ", version, original filename, original size, code table, CRC32
HZ-F-005	Verify CRC32 of decompressed data against the header
HZ-F-006	Display statistics: original size, compressed size, ratio (%), time taken
HZ-F-007	Correctly handle empty (0-byte) files and single-symbol files
HZ-F-008	Report errors for invalid input with a clear message and non-zero exit code
HZ-F-009	Prompt before overwriting an existing output file (skip prompt with -f)
HZ-NF-001	Performance: compress and decompress a 10 MB text file in ≤ 3 s each on the reference machine
HZ-NF-002	Correctness: decompressed output is byte-identical to the original in 100% of cases
HZ-NF-003	Compression: $\geq 30\%$ average size reduction on the Canterbury Corpus English text files
HZ-NF-004	Memory: peak memory ≤ 64 MB for inputs up to 1 GB (streamed in chunks)
HZ-NF-005	Portability: builds with zero errors and passes all tests on Ubuntu 22.04 (GCC 11+) and Windows 10/11 (MinGW-w64)
HZ-NF-006	Usability: -h shows usage; every error message states the cause
HZ-SEC-001	Validate magic, version and header fields before allocating memory; reject malformed archives
HZ-SEC-002	Abort decompression if output would exceed the original size stored in the header
HZ-SEC-003	Sanitize the stored filename: strip directories, reject absolute paths and ".."
HZ-SEC-004	No memory-safety errors or leaks (AddressSanitizer / Valgrind clean)

4. Features Not to be Tested

- **Interoperability with PKZIP, WinZip, 7-Zip or other standard ZIP tools:** HuffZip uses its own .huf format; standard ZIP uses DEFLATE (LZ77 + Huffman), which is outside project scope.
- **Graphical user interface:** the product is command-line only.
- **Encryption and password protection:** not part of the v1.0 requirements.
- **Multi-file and directory archiving:** planned future enhancement.
- **Operating system, filesystem and C/C++ standard library internals:** assumed correct and tested by their vendors.

5. Test Approach / Strategy

Levels:

- **Unit tests (GoogleTest):** frequency counting, tree construction, code table generation, bit writer/reader, header serialization, CRC32 against known values.
- **Integration tests:** Encoder ↔ Header Writer ↔ File I/O; Header Reader ↔ Decoder ↔ Integrity Checker.
- **System tests:** end-to-end CLI tests through shell scripts (compress → decompress → compare).
- **Acceptance tests:** demonstration of all SRS features to the course instructor.

Types:

- **Functional testing:** all HZ-F requirements.
- **Boundary and negative testing:** empty files, single-symbol files, missing files, invalid options.
- **Regression testing:** full automated suite re-run on every commit / pull request.
- **Performance testing:** execution time and peak memory.
- **Security testing:** see Section 5.1.
- **Portability and usability testing:** Linux and Windows builds; help and error messages.

Test design techniques: equivalence partitioning (text / binary / incompressible inputs), boundary value analysis (0 bytes, 1 byte, 1 symbol, 1 GB), error guessing (corrupted and malicious archives), and round-trip (oracle) testing using byte-wise comparison.

Entry Criteria: SRS and SAD baselined; code compiles without errors on Linux; unit tests for core modules exist; test data generated; test environment ready.

Exit Criteria: 100% of planned test cases executed; 100% of High-priority cases passed; 0 Critical/High defects open; every SRS requirement covered by at least one passed test case in the RTM; ≥ 80% line coverage on core modules (gcov/lcov).

5.1 Security Validation

- **Archive header validation (HZ-SEC-001):** feed invalid magic numbers, wrong version, truncated headers and absurd size / table-length fields; confirm rejection before any large allocation (TC-SEC-02).
- **Integrity / tampering (HZ-F-005):** flip bits in the data section and confirm CRC32 mismatch is reported and partial output deleted (TC-SEC-01).
- **Decompression-bomb protection (HZ-SEC-002):** crafted archive whose payload exceeds the declared size must be aborted (TC-SEC-03).
- **Path traversal (HZ-SEC-003):** stored filenames containing "..", "/" or absolute paths must be sanitized or rejected (TC-SEC-04).
- **Memory safety (HZ-SEC-004):** run all tests under AddressSanitizer, UndefinedBehaviorSanitizer and Valgrind (TC-MEM-01).
- **Fuzz testing:** run libFuzzer or AFL++ on the decompress entry point for at least 30 minutes; 0 crashes or hangs allowed.
- **Static analysis:** cppcheck and compiler warnings (-Wall -Wextra) with 0 high-severity findings.

6. Test Environment

Hardware (reference machine): <e.g., Intel Core i5 (8th gen or later), 8 GB RAM, SSD> – record exact specs; all performance figures are measured on this machine.

Software: Ubuntu 22.04 LTS with GCC 11+; Windows 10/11 with MinGW-w64; CMake 3.20+; HuffZip v1.0 build.

Tools: GoogleTest + CTest (unit/integration), Bash / PowerShell scripts (system tests), cmp / fc /b (output comparison), GNU time and Valgrind (performance and memory), AddressSanitizer / UBSan, libFuzzer or AFL++ (fuzzing), gcov / lcov (coverage), cppcheck (static analysis), GitHub Actions (CI), GitHub Issues (defect tracking).

Test Data: Canterbury Corpus text files; sample PNG and executable files; generated files (empty, single-symbol, 10 MB text, 1 GB binary) created by a gen_data script rather than stored in Git; crafted malicious archives (corrupted, truncated, bomb, path-traversal).

7. Test Schedule

Milestones (adjust to the team's sprint plan):

Activity	Date
Test plan and test case design	05-Oct-2026 – 07-Oct-2026
Test environment and test data setup	08-Oct-2026 – 10-Oct-2026
Unit testing (alongside each sprint)	10-Oct-2026 onwards
Integration and system test execution	<Sprint 2 end date>
Security, performance and portability testing	<Sprint 3 dates>
Acceptance testing / demo	<Final review date>
Test Summary Report	<Final submission date>

8. Test Deliverables

- Test Plan (this document)
- Test Case Specifications (Section 13.2)
- Automated test scripts (GoogleTest suites, shell scripts) in the tests/ folder of the repository
- Test data and the gen_data script
- Test execution logs and CI run reports
- Defect reports (GitHub Issues)
- Coverage report (lcov HTML)
- Test Summary Report

9. Roles and Responsibilities

Role	Name (SRN)	Responsibility
QA Lead	Pranavi Sistla (PES1UG24CS338)	Prepare test plan, maintain RTM, coordinate execution, prepare Test Summary Report
Test Engineer – Functional	Pavan Kumar H K (PES1UG24CS319)	Design and execute functional, boundary and negative test cases; log defects

Role	Name (SRN)	Responsibility
Test Engineer – Security & Performance	Rachit Upadhyay (PES1UG24CS355)	Security validation, fuzzing, sanitizer runs, performance and memory tests
Developer / CI Owner	Inchara K Kuppal (PES1UG25CS818)	Write unit tests, maintain CI pipeline, fix and triage defects
Product Owner	<Course faculty>	Approve test results and sign off release readiness

10. Risks and Mitigation

Risk	Mitigation
Huffman edge cases (empty file, single symbol) found late	Write unit tests for them in the first sprint (TC-EDGE-01, TC-EDGE-02)
Windows text-mode I/O corrupts binary data (\r\n translation)	Open all files in binary mode; cross-platform test TC-PORT-01
Large test files (1 GB) too big for the repository	Generate them with the gen_data script; exclude via .gitignore
Performance varies between team members' machines	Fix one reference machine and record its specification
Delay in stable builds due to academic workload	Smoke test on every commit via CI; prioritise High-priority test cases

11. Assumptions & Dependencies

- SRS v1.0 is baselined and its requirement IDs match those in Section 3.
- The reference machine and both operating systems are available to the team.
- Canterbury Corpus files are downloadable and unchanged.
- GoogleTest, Valgrind, sanitizers and fuzzing tools install successfully on the test machines.

12. Suspension & Resumption Criteria

Suspend testing if:

- The build fails to compile on the primary platform.
- The basic round-trip test (TC-COMP-01) fails, since it blocks almost all other tests.
- Blocking defects prevent execution of more than 30% of test cases.

Resume testing when:

- The blocking defects are fixed and the smoke test (TC-COMP-01) passes.
- The build is stable and the environment is restored.

13. Test Case Management & Traceability

Test cases are stored in this document and automated in the repository's tests/ folder. Each test case references one or more SRS requirement IDs. The matrix below is also copied into Appendix C (RTM) of the SRS.

13.1 Requirements Traceability Matrix

Requirement ID	Requirement	Test Case ID(s)
HZ-F-001	Compress a single input file into a .huf archive using Huffman coding	TC-COMP-01, TC-COMP-02
HZ-F-002	Decompress a .huf archive and restore the original file	TC-COMP-01, TC-COMP-02
HZ-F-003	Build the byte-frequency table and Huffman tree / code table	TC-COMP-01, TC-EDGE-02
HZ-F-004	Write archive header: magic "HUFZ", version, original filename, original size, code table, CRC32	TC-HDR-01
HZ-F-005	Verify CRC32 of decompressed data against the header	TC-SEC-01
HZ-F-006	Display statistics: original size, compressed size, ratio (%), time taken	TC-STAT-01
HZ-F-007	Correctly handle empty (0-byte) files and single-symbol files	TC-EDGE-01, TC-EDGE-02
HZ-F-008	Report errors for invalid input with a clear message and non-zero exit code	TC-ERR-01
HZ-F-009	Prompt before overwriting an existing output file (skip prompt with -f)	TC-ERR-02
HZ-NF-001	Performance: compress and decompress a 10 MB text file in ≤ 3 s each on the reference machine	TC-PERF-01
HZ-NF-002	Correctness: decompressed output is byte-identical to the original in 100% of cases	TC-COMP-01, TC-COMP-02
HZ-NF-003	Compression: $\geq 30\%$ average size reduction on the Canterbury Corpus English text files	TC-RATIO-01
HZ-NF-004	Memory: peak memory ≤ 64 MB for inputs up to 1 GB (streamed in chunks)	TC-PERF-02
HZ-NF-005	Portability: builds with zero errors and passes all tests on Ubuntu 22.04 (GCC 11+) and Windows 10/11 (MinGW-w64)	TC-PORT-01
HZ-NF-006	Usability: -h shows usage; every error message states the cause	TC-ERR-01, TC-USE-01
HZ-SEC-001	Validate magic, version and header fields before allocating memory; reject malformed archives	TC-SEC-01, TC-SEC-02
HZ-SEC-002	Abort decompression if output would exceed the original size stored in the header	TC-SEC-03
HZ-SEC-003	Sanitize the stored filename: strip directories, reject absolute paths and ".."	TC-SEC-04

Requirement ID	Requirement	Test Case ID(s)
HZ-SEC-004	No memory-safety errors or leaks (AddressSanitizer / Valgrind clean)	TC-MEM-01

13.2 Test Case Specifications

Total test cases: 18 (functional: 8, security: 5, non-functional: 5). Actual Result and Status are filled in during execution.

Test Case ID	TC-COMP-01
Requirement ID(s)	HZ-F-001, HZ-F-002, HZ-F-003, HZ-NF-002
Title	Round-trip of a plain text file
Test Type	Functional / System
Priority	High
Preconditions	huffzip binary built; alice29.txt (152,089 bytes, Canterbury Corpus) in working directory
Test Steps	1. Run: huffzip -c alice29.txt -o alice.huf 2. Run: huffzip -d alice.huf -o out.txt 3. Run: cmp alice29.txt out.txt
Test Data	alice29.txt
Expected Result	Both commands exit with code 0; alice.huf is smaller than alice29.txt; cmp reports no differences.
Actual Result	(fill after execution)
Status (Pass/Fail)	(fill after execution)

Test Case ID	TC-COMP-02
Requirement ID(s)	HZ-F-001, HZ-F-002, HZ-NF-002
Title	Round-trip of a binary file
Test Type	Functional / System
Priority	High
Preconditions	Binary test files available
Test Steps	1. Compress sample.png and a compiled executable 2. Decompress each archive 3. Compare with cmp (Linux) / fc /b (Windows)
Test Data	sample.png, an ELF/EXE binary
Expected Result	Exit code 0 for all steps; outputs byte-identical to originals.
Actual Result	(fill after execution)
Status (Pass/Fail)	(fill after execution)

Test Case ID	TC-HDR-01
Requirement ID(s)	HZ-F-004
Title	Archive header contents are correct
Test Type	Functional / Unit
Priority	High
Preconditions	alice.huf created by TC-COMP-01
Test Steps	1. Run: xxd alice.huf head 2. Read header via the unit-test helper read_header()
Test Data	alice.huf
Expected Result	First 4 bytes are "HUFZ"; version = 1; stored filename = "alice29.txt"; original size = 152089; CRC32 field equals CRC32 of alice29.txt.
Actual Result	(fill after execution)
Status (Pass/Fail)	(fill after execution)

Test Case ID	TC-EDGE-01
Requirement ID(s)	HZ-F-007
Title	Empty (0-byte) input file
Test Type	Functional / Boundary
Priority	High
Preconditions	empty.txt created with: touch empty.txt
Test Steps	1. huffzip -c empty.txt -o empty.huf 2. huffzip -d empty.huf -o empty_out.txt 3. Check size of empty_out.txt
Test Data	empty.txt (0 bytes)
Expected Result	No crash; archive contains header only; empty_out.txt is 0 bytes; exit code 0.
Actual Result	(fill after execution)
Status (Pass/Fail)	(fill after execution)

Test Case ID	TC-EDGE-02
Requirement ID(s)	HZ-F-007, HZ-F-003
Title	File with a single repeated symbol
Test Type	Functional / Boundary
Priority	High
Preconditions	single.txt containing 10,000 × 'A'

Test Steps	1. Compress single.txt 2. Decompress the archive 3. cmp with original
Test Data	single.txt
Expected Result	Huffman tree with one leaf is handled (code length 1); round-trip byte-identical; no crash or infinite loop.
Actual Result	(fill after execution)
Status (Pass/Fail)	(fill after execution)

Test Case ID	TC-STAT-01
Requirement ID(s)	HZ-F-006
Title	Compression statistics are displayed correctly
Test Type	Functional
Priority	Medium
Preconditions	alice29.txt available
Test Steps	1. huffzip -c alice29.txt -o alice.huf 2. Note printed stats 3. Compare with sizes from ls -l / stat
Test Data	alice29.txt
Expected Result	Output shows original size, compressed size, ratio (%) and time; sizes match ls -l exactly; ratio = $(1 - \text{compressed/original}) \times 100$, rounded to 2 decimals.
Actual Result	(fill after execution)
Status (Pass/Fail)	(fill after execution)

Test Case ID	TC-ERR-01
Requirement ID(s)	HZ-F-008, HZ-NF-006
Title	Input file does not exist
Test Type	Negative
Priority	High
Preconditions	missing.txt does not exist
Test Steps	1. huffzip -c missing.txt -o m.huf 2. Check exit code (echo \$?) 3. Check whether m.huf exists
Test Data	missing.txt
Expected Result	Message "Error: input file 'missing.txt' not found"; exit code non-zero (2); m.huf is not created.
Actual Result	(fill after execution)

Status (Pass/Fail)	(fill after execution)
--------------------	------------------------

Test Case ID	TC-ERR-02
Requirement ID(s)	HZ-F-009
Title	Overwrite protection for existing output
Test Type	Functional / Negative
Priority	Medium
Preconditions	out.huf already exists
Test Steps	1. huffzip -c alice29.txt -o out.huf, answer 'n' 2. Repeat, answer 'y' 3. Repeat with -f flag
Test Data	alice29.txt, existing out.huf
Expected Result	Step 1: file unchanged (checksum same); Step 2: file overwritten; Step 3: overwritten without a prompt.
Actual Result	(fill after execution)
Status (Pass/Fail)	(fill after execution)

Test Case ID	TC-SEC-01
Requirement ID(s)	HZ-F-005, HZ-SEC-001
Title	Corrupted archive detected by CRC32
Test Type	Security / Negative
Priority	High
Preconditions	alice.huf from TC-COMP-01
Test Steps	1. Copy to bad.huf and flip one byte in the data section (e.g., offset 500) with a hex editor or script 2. huffzip -d bad.huf -o bad_out.txt
Test Data	bad.huf
Expected Result	Message "Error: CRC mismatch – archive is corrupted"; non-zero exit code; partial output file deleted.
Actual Result	(fill after execution)
Status (Pass/Fail)	(fill after execution)

Test Case ID	TC-SEC-02
Requirement ID(s)	HZ-SEC-001
Title	Invalid magic number / truncated header
Test Type	Security / Negative

Priority	High
Preconditions	Crafted files: notarchive.huf (renamed .txt), truncated.huf (first 6 bytes of a valid archive)
Test Steps	1. huffzip -d notarchive.huf 2. huffzip -d truncated.huf
Test Data	notarchive.huf, truncated.huf
Expected Result	Both rejected with "Error: not a valid HuffZip archive"; no crash; no large memory allocation; non-zero exit code.
Actual Result	(fill after execution)
Status (Pass/Fail)	(fill after execution)

Test Case ID	TC-SEC-03
Requirement ID(s)	HZ-SEC-002
Title	Decompression-bomb protection
Test Type	Security
Priority	High
Preconditions	bomb.huf: valid archive with header original-size edited to 100 bytes but payload decoding to far more
Test Steps	1. huffzip -d bomb.huf -o bomb_out.txt 2. Check size of any output
Test Data	bomb.huf
Expected Result	Decompression aborts once 100 bytes are written; error reported; output removed; non-zero exit code.
Actual Result	(fill after execution)
Status (Pass/Fail)	(fill after execution)

Test Case ID	TC-SEC-04
Requirement ID(s)	HZ-SEC-003
Title	Path-traversal filename in archive
Test Type	Security
Priority	High
Preconditions	evil.huf with stored filename "../tmp/evil.txt"; second archive with "/etc/evil"
Test Steps	1. huffzip -d evil.huf (no -o, so stored name is used) 2. Repeat with absolute-path archive 3. Check filesystem outside the working directory
Test Data	evil.huf, abs.huf

Expected Result	Name is sanitized to "evil.txt" in the current directory, or the archive is rejected; no file is written outside the working directory.
Actual Result	(fill after execution)
Status (Pass/Fail)	(fill after execution)

Test Case ID	TC-MEM-01
Requirement ID(s)	HZ-SEC-004
Title	Memory safety under sanitizers
Test Type	Security / Non-functional
Priority	High
Preconditions	Debug build with -fsanitize=address,undefined; Valgrind installed
Test Steps	1. Run full unit + system suite under the ASan build 2. Run TC-COMP-01 and TC-SEC-02 under valgrind --leak-check=full
Test Data	Full test data set
Expected Result	0 ASan/UBSan errors; Valgrind reports 0 errors and "definitely lost: 0 bytes".
Actual Result	(fill after execution)
Status (Pass/Fail)	(fill after execution)

Test Case ID	TC-PERF-01
Requirement ID(s)	HZ-NF-001
Title	Compression / decompression time for 10 MB
Test Type	Performance
Priority	High
Preconditions	Reference machine (see Section 6); text10mb.txt generated by gen_data script
Test Steps	1. time huffzip -c text10mb.txt -o t.huf (5 runs) 2. time huffzip -d t.huf -o t.txt (5 runs) 3. Average wall-clock time
Test Data	text10mb.txt (10,485,760 bytes)
Expected Result	Average compression time ≤ 3 s and average decompression time ≤ 3 s.
Actual Result	(fill after execution)
Status (Pass/Fail)	(fill after execution)

Test Case ID	TC-PERF-02
Requirement ID(s)	HZ-NF-004
Title	Peak memory for a 1 GB input

Test Type	Performance
Priority	Medium
Preconditions	big1gb.bin generated; GNU time available
Test Steps	1. /usr/bin/time -v huffzip -c big1gb.bin -o big.huf 2. Read "Maximum resident set size" 3. Repeat for decompression
Test Data	big1gb.bin
Expected Result	Maximum resident set size $\leq$ 65,536 KB (64 MB) for both operations.
Actual Result	(fill after execution)
Status (Pass/Fail)	(fill after execution)

Test Case ID	TC-RATIO-01
Requirement ID(s)	HZ-NF-003
Title	Compression ratio on English text
Test Type	Non-functional
Priority	Medium
Preconditions	Canterbury Corpus text files downloaded
Test Steps	1. Compress alice29.txt, asyoulik.txt, lcet10.txt, plrabn12.txt 2. Record reduction % for each 3. Compute the average
Test Data	Canterbury Corpus (4 files)
Expected Result	Average size reduction $\geq$ 30%; results recorded in the Test Summary Report.
Actual Result	(fill after execution)
Status (Pass/Fail)	(fill after execution)

Test Case ID	TC-PORT-01
Requirement ID(s)	HZ-NF-005
Title	Cross-platform build and run
Test Type	Non-functional
Priority	Medium
Preconditions	Ubuntu 22.04 (GCC 11+) and Windows 10/11 (MinGW-w64) machines
Test Steps	1. Build with -Wall -Wextra on both platforms 2. Run the full test suite on both 3. Compress on Linux, decompress on Windows, compare
Test Data	alice29.txt, sample.png

Expected Result	Zero compile errors on both; all tests pass; cross-platform archive decompresses byte-identically (verifies binary-mode file I/O).
Actual Result	(fill after execution)
Status (Pass/Fail)	(fill after execution)

Test Case ID	TC-USE-01
Requirement ID(s)	HZ-NF-006
Title	Help and invalid-option handling
Test Type	Usability
Priority	Low
Preconditions	huffzip built
Test Steps	1. huffzip -h 2. huffzip -x file.txt 3. huffzip (no arguments)
Test Data	—
Expected Result	Step 1 prints usage with all options, exit 0; Steps 2–3 print "Unknown option" / usage hint, non-zero exit code.
Actual Result	(fill after execution)
Status (Pass/Fail)	(fill after execution)

14. Test Metrics & Reporting

Metrics collected:

- % test cases executed
- % passed / failed
- Defect count by severity and defect density (defects per KLOC)
- Defect aging (days open)
- Requirement coverage (% of SRS requirements with a passed test)
- Code coverage (% lines and branches, gcov/lcov)
- Benchmark results: compression ratio, time and peak memory per test file

Reports:

- CI status on every commit (GitHub Actions)
- Weekly execution status at the end of each sprint
- Final Test Summary Report

15. Approvals

Role	Name	Signature / Date
QA Lead		
Dev Lead		
Product Owner		